



- Name : Harshvardhan Patil
- Branch : AI&DS
- Roll No : SY B 23
- Batch : SY B2

AIM

Data Visualization with Matplotlib: Data visualization on a dataset using matplotlib and seaborn libraries • Scatter plot • Line plot • Bar plot • Histogram • Box plot • Pair plot

THEORY

1.What is the importance of data visualization in data analysis? How does visualization help in exploratory data analysis (EDA)?

Data visualization helps in quickly understanding complex data, identifying patterns, trends, outliers, and relationships not clear from raw data. In EDA, it's crucial for understanding data distribution, detecting outliers, visualizing variable relationships, spotting trends, validating assumptions, and effectively communicating insights.

2.What is the difference between Matplotlib and other visualization libraries?

Matplotlib is a low-level plotting library that gives users complete control over every part of a graph, such as axes, labels, colors, and layout. Because of this, it is very flexible and widely used in research and academic work, but it requires more code and can be harder for beginners.

In comparison, libraries like Seaborn, Plotly, and Bokeh are high-level libraries designed for ease of use. Seaborn is built on Matplotlib and simplifies statistical plots with better default styles. Plotly and Bokeh go a step further by providing interactive features like zooming, hovering, and dynamic dashboards, which are useful for web applications and real-time data visualization.

3.What is the purpose of the pyplot module in Matplotlib?

The matplotlib.pyplot module is used to provide a simple and user-friendly interface for creating plots and graphs. It works like MATLAB-style plotting, where functions are used to quickly generate figures, axes, and visualizations.

The main purpose of pyplot is to:

Create different types of plots (line, bar, scatter, etc.) Manage figure and axes objects Add labels, titles, and legends Display or save graphs

4.Explain the process of creating a basic line plot

Creating a basic line plot in Matplotlib involves a few simple steps:

Import pyplot module

```
import matplotlib.pyplot as plt
```

Prepare data

```
x = [1, 2, 3, 4]
y = [10, 20, 25, 30]
```

Create the plot

```
plt.plot(x, y)
```

Add labels and title

```
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Basic Line Plot")
```

Display the plot

```
plt.show()
```

5.Why is customization important in data visualization? What customization options are available in Matplotlib?

Customization is important because it makes visualizations:

Clear and easy to understand Visually appealing More informative and meaningful Suitable for presentations and reports

Without customization, graphs may look confusing or fail to convey the correct message.

In Matplotlib, several customization options are available:

1.Titles and Labels

```
plt.title(), plt.xlabel(), plt.ylabel()
```

2.Colors and Styles

Change line colors, markers, line styles

Example: `plt.plot(x, y, color='red', linestyle='--')`

3.Legends

`plt.legend()` to describe multiple data lines

4 Grid

`plt.grid()` for better readability

5.Axis Limits

`plt.xlim()`, `plt.ylim()` to control scale

6.Markers

Add points using markers like 'o', '^', etc.

7.Figure Size

`plt.figure(figsize=(width, height))`

```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('/content/cleaned_employee_data.csv')
print(df.head())
sns.set(style="whitegrid")

# 1. Scatter Plot
plt.figure()
plt.scatter(df['Age'], df['Salary'])
plt.xlabel('Age')
plt.ylabel('Salary')
plt.title('Scatter Plot (Age vs Salary)')
plt.show()

# 2. Line Plot
plt.figure(figsize=(4,3))
plt.plot(df['Age'], df['Salary'])
plt.xlabel('Age')
```

```

plt.ylabel('Salary')
plt.title('Line Plot (Age vs Salary)')
plt.show()

# 3. Bar Plot (Seaborn)
plt.figure(figsize=(4,3))
sns.barplot(x='Department', y='Salary', data=df)
plt.title('Bar Plot (Department vs Salary)')
plt.xticks(rotation=45)
plt.show()

# 4. Histogram
plt.figure(figsize=(4,3))
plt.hist(df['Salary'], bins=10)
plt.xlabel('Salary')
plt.ylabel('Frequency')
plt.title('Histogram of Salary')
plt.show()

# 5. Box Plot (Seaborn)
plt.figure(figsize=(4,3))
sns.boxplot(x='Department', y='Salary', data=df)
plt.title('Box Plot (Department vs Salary)')
plt.xticks(rotation=45)
plt.show()

# 6. Pair Plot (Seaborn)

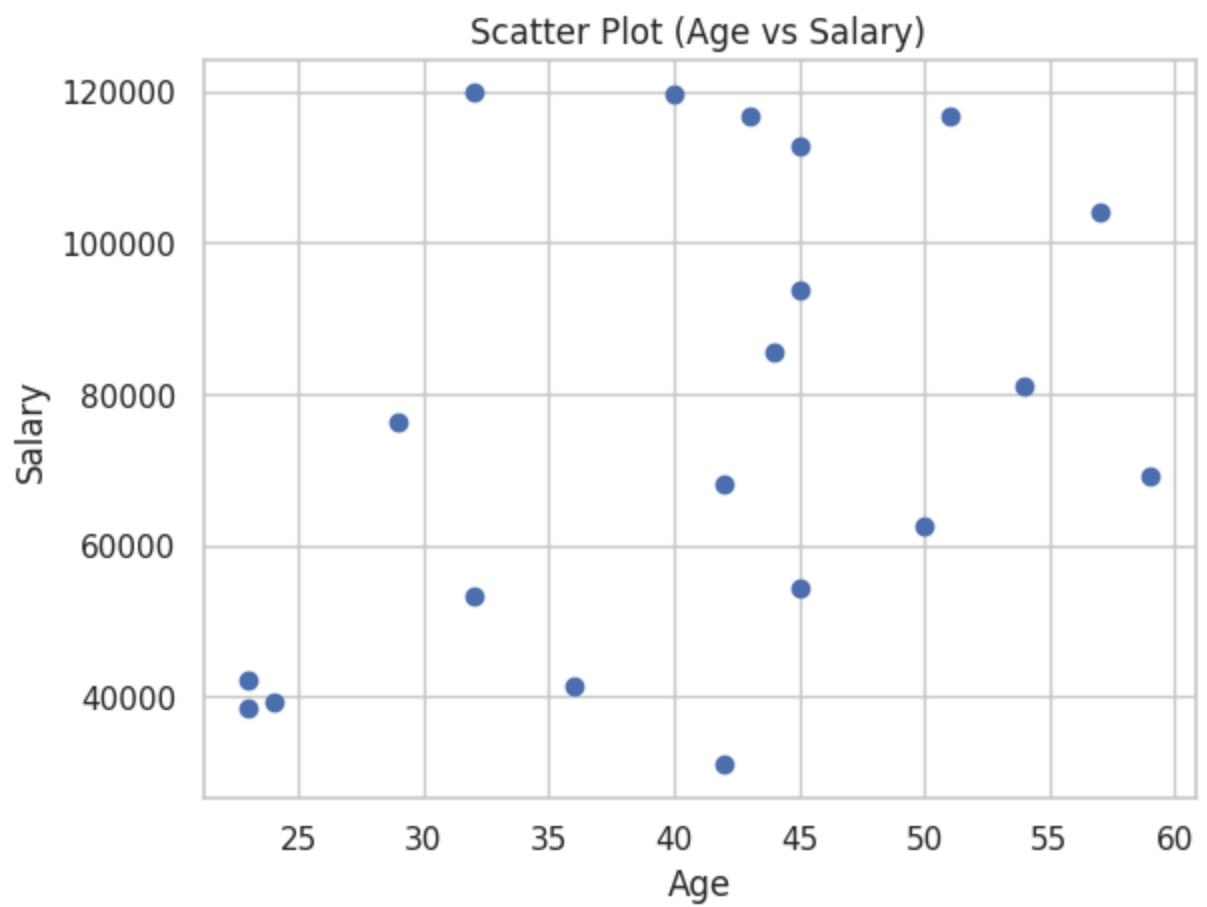
sns.pairplot(df,height=1.3)
plt.show()

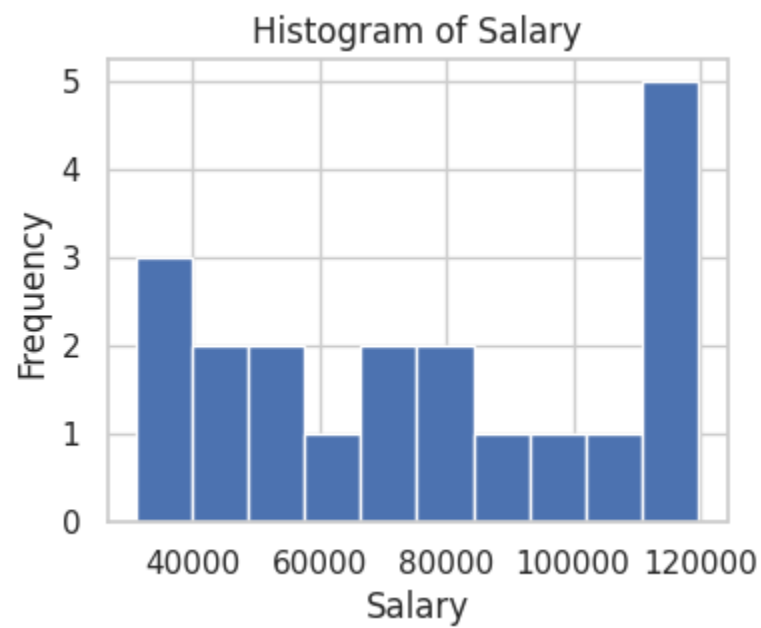
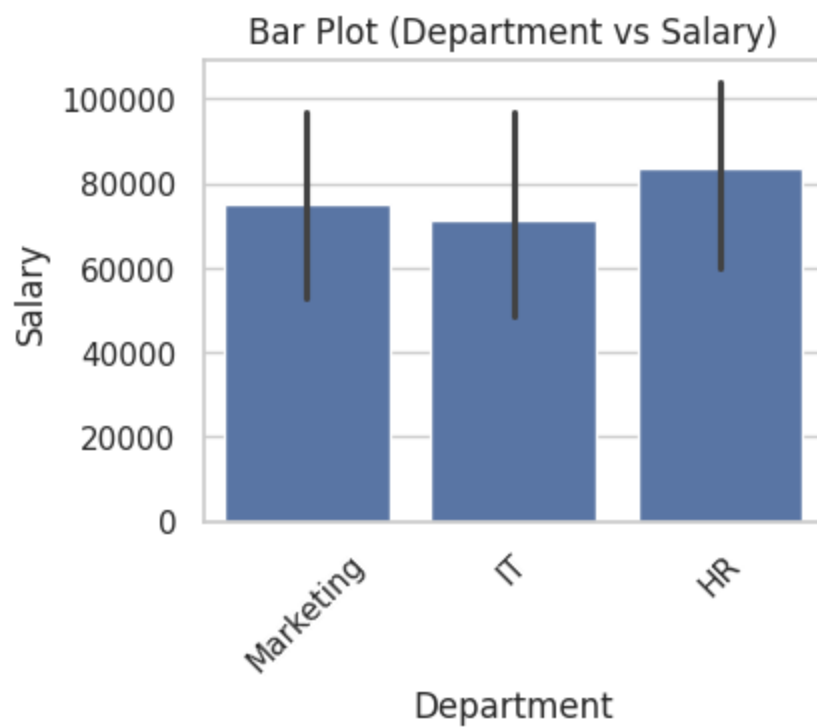
```

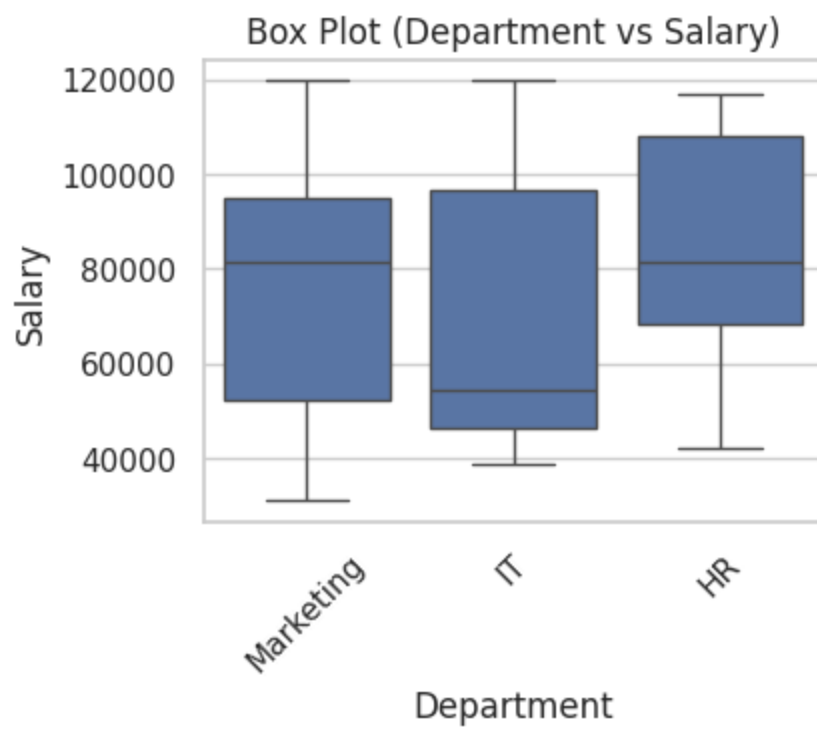
	Employee_ID	Name	Age	Gender	Department	Join_Date	Salary \
0	101	Alice	50	Female	Marketing	2009-01-18	62606.000000
1	102	Bob	36	Female	Marketing	2012-01-01	41534.000000
2	103	Charlie	29	Female	IT	2007-02-15	76338.368421
3	104	David	42	Male	Marketing	2015-12-29	31016.000000
4	105	Eve	40	Female	IT	2005-02-04	119789.000000

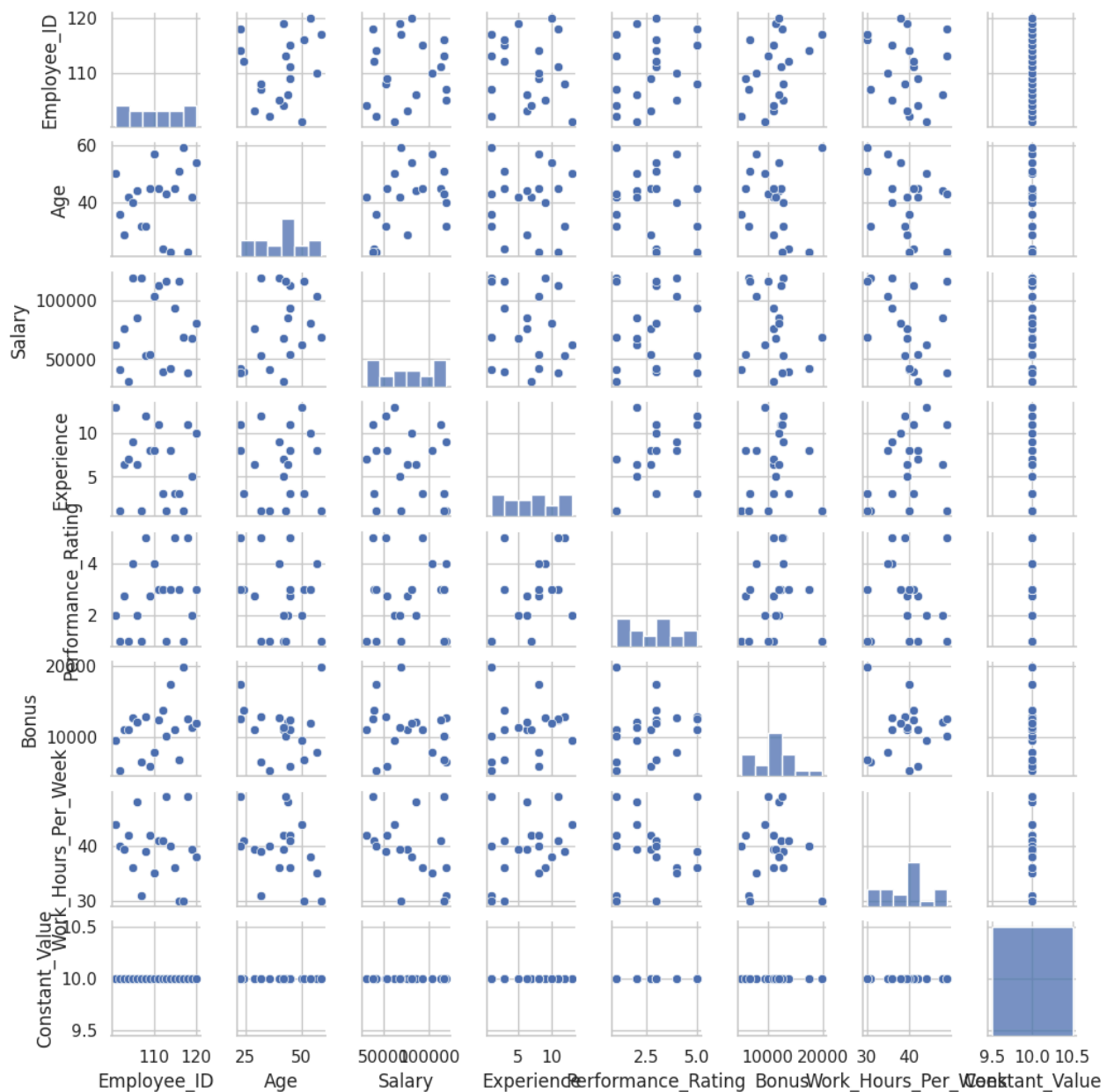
	Experience	Performance_Rating	Bonus	Work_Hours_Per_Week \
0	13.000000	2.000000	9488.000000	44.0
1	1.000000	1.000000	5206.000000	40.0
2	6.388889	2.722222	11083.388889	39.5
3	7.000000	1.000000	10977.000000	42.0
4	9.000000	4.000000	12721.000000	36.0

	Experience/Fresher	Constant_Value
0	Experienced	10
1	Fresher	10
2	Fresher	10
3	Experienced	10
4	Experienced	10









CONCLUSION

The practical session successfully demonstrated various data visualization techniques using Matplotlib and Seaborn on the cleaned_employee_data.csv dataset.

We were able to:

Load the dataset into a pandas DataFrame. Generate a Scatter Plot to visualize the relationship between 'Age' and 'Salary'. Create a Line Plot to show trends between 'Age' and 'Salary'.

Produce a Bar Plot to compare 'Salary' across different 'Department' categories.

Display a Histogram to understand the distribution of 'Salary'.

Generate a Box Plot to show the distribution and potential outliers of 'Salary' within each 'Department'.

Utilize a Pair Plot to visualize pairwise relationships and distributions for all numerical columns in the DataFrame.

This practical session effectively showcased how these different plot types can be used to explore and understand patterns, distributions, and relationships within a dataset.